VHDL Programs

VHDL code for **and gate** with **data flow modeling**

```
library ieee;
use ieee.std_logic_1164.all;
entity andgate is
port(
   x,y: in std_logic;
   z: out std_logic
);
end andgate;
architecture arc_andgate of andgate is
begin
z<= x and y;
end arc_andgate;
```

VHDL code for **half adder** with **data flow modeling**

```
library ieee;
use ieee.std_logic_1164.all;
entity half_adder is
port(
   a,b: in std_logic;
   c,s: out std_logic
);
end half_adder;
architecture arc_half_adder of half_adder is
begin
c<= a and b;
s<= a xor b;
end arc_half_adder;
```

VHDL code for **full adder** with **data flow modeling**

```
library ieee;
use ieee.std_logic_1164.all;
entity full_adder is
port(
   a,b,c: in std_logic;
   carry,sum: out std_logic
);
end full_adder;
architecture arc_full_adder of full_adder is
signal x,y,z: std_logic;
begin
```

```
x<= a xor b;
sum<= x xor c;
y<= x and c;
z<= a and b;
carry<= y or z;
end arc_full_adder;
```

VHDL code for **half adder** with **structural modeling**

```
library ieee;
use ieee.std_logic_1164.all;

entity and_gate is
port(
   a,b: in std_logic;
   c: out std_logic
);
end and_gate;
architecture arc_and_gate of and_gate is
begin
c<= a and b;
end arc_and_gate;

library ieee;
use ieee.std_logic_1164.all;

entity xor_gate is
port(
   a,b: in std_logic;
   c: out std_logic
);
end xor_gate;

architecture arc_xor_gate of xor_gate is
begin
c<= a xor b;
end arc_xor_gate;

library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
port(
   a,b: in std_logic;
   carry,sum: out std_logic
```

```
);
end half_adder;

architecture arc_half_adder of half_adder is
component and_gate
port(
   a,b: in std_logic;
   c: out std_logic
);
end component;
component xor_gate
port(
   a,b: in std_logic;
   c: out std_logic
);
end component;
begin
   U0: and_gate port map (
       a => a,
       b => b,
       c => carry
     );
   U1: xor_gate port map(
      a => a,
      b => b,
      c => sum
   );
end arc_half_adder;
```

Write a VHDL program for **4:1 multiplexer** with **behavioral modeling**.

```
library ieee;
use ieee.std_logic_1164.all;

entity multiplexer4_1 is
port(i0 , i1, i2, i3: in std_logic;
        sel : in std_logic_vector(1 downto 0);
        bitout: out std_logic);
end multiplexer4_1;

architecture behavioral of multiplexer4_1 is
begin

process(i0,i1,i2,i3,sel)
begin
```

```
case sel is
  when "00" => bitout <= i0;
  when "01" => bitout <= i1;
  when "10" => bitout <= i2;
  when others => bitout <= i3;
end case;
end process;

end behavioral;
```

VHDL program for **1:8 demux** with **behavioral modeling**

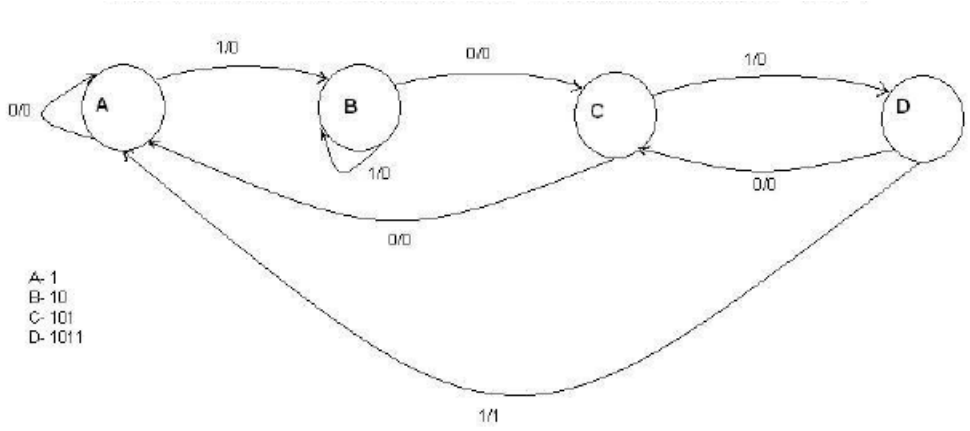
```
library ieee;
use ieee.std_logic_1164.all;

entity demux_1_8 is
port( i : in std_logic;
        sel : in std_logic_vector(0 to 2);
        bitout :  out std_logic_vector (0 to 7));
end demux_1_8;

architecture behavioral of demux_1_8 is
begin

process(i, sel)
begin
    bitout <= "00000000";
case sel is
  when "000" => bitout(0) <= i;
  when "001" => bitout(1) <= i;
  when "010" => bitout(2) <= i;
  when "011" => bitout(3) <= i;
  when "100" => bitout(4) <= i;
  when "101" => bitout(5) <= i;
  when "110" => bitout(6) <= i;
  when "111" => bitout(7) <= i;
  when others => bitout <= "00000000";
end case;
end process;

end behavioral;
```

VHDL code for **4-bit full adder** with **structural modeling**

```
Library ieee;
use ieee.std_logic_1164.all;

entity fulladder is
port
(  sum, carry : out std_logic;
   a,b,c : in std_logic
);
end fulladder;

architecture arc_full_adder of fulladder is
signal x, y, z : std_logic;
begin
   x<= a xor b;
   sum<= x xor c;
   y<= x and c;
   z<= a and b;
   carry<= y or z;
end arc_full_adder;

library ieee;
use ieee.std_logic_1164.all;

entity BPA4 is
port
(  A,B: in std_logic_vector(3 downto 0);
   Sum_bpa: out std_logic_vector(3 downto 0);
   Cin : in std_logic;
   Cout: out std_logic
);
end BPA4;

architecture arc_BPA4 of BPA4 is
component fulladder
port
(  sum, carry : out std_logic;
   a,b,c: in std_logic
);

End component;
signal c1, c2, c3 : std_logic;
begin
U0: fulladder port map(
```

```
   a=>A(0),
   b=>B(0),
   c=>Cin,
   sum=>sum_bpa(0),
   carry=>c1);
U1: fulladder port map(
   a=>A(1),
   b=>B(1),
   c=>C1,
   sum=>sum_bpa(1),
   carry=>c2);
U2: fulladder port map(
   a=>A(2),
   b=>B(2),
   c=>C2,
   sum=>sum_bpa(2),
   carry=>c3);
U3: fulladder port map(
   a=>A(3),
   b=>B(3),
   c=>c3,
   sum=>sum_bpa(3),
   carry=>Cout);
end arc_BPA4;
```

Write VHDL code for **sequence "1011" detector** without **overlapping condition**.



```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity seq_detector is
port( clk: in std_logic;
    reset: in std_logic;
    seq: in std_logic;
    detected: out std_logic
    );
end seq_detector;

architecture behavioral of seq_detector is

type state_type is (A,B,C,D);
signal state: state_type := A;

begin

process(clk, reset)

begin
    if(reset = '1') then
    detected <= '0';
    state <= A;
    elsif (rising_edge(clk)) then

    case state is

    when A =>
    detected <= '0';
    if (seq = '0') then
    state <= A;
    else
      state <= B;
    end if;

    when B =>
    if (seq = '0') then
    state <= C;
    else
      state <= B;
    end if;

    when C =>
    if (seq = '0') then
```

```
    state <= A;
    else
      state <= D;
    end if;

    when D =>
    if (seq = '0') then
    state <= C;
    else
      state <= A;
      detected <= '1';
    end if;

    when others =>
    null;

    end case;

    end if;

end process;
end behavioral;
```

Write VHDL code for **octal to binary encoder**

```
library ieee;
use ieee.std_logic_1164.all;

entity encoder_8_3 is
port( i : in std_logic_vector(7 downto 0);
        bit_out : out std_logic_vector(0 to 2));
end encoder_8_3;

architecture arch_encoder of encoder_8_3 is
begin

process(i)
begin
    bit_out(0) <= i(1) or i(3) or i(5) or i(7);
    bit_out(1) <= i(2) or i(3) or i(6) or i(7);
    bit_out(2) <= i(4) or i(5) or i(6) or i(7);
end process;

end arch_encoder;
```